The Abeselom ASIC-Direct 50 (AAD-50):

A Firmware-Enforced, 50-Cycle Sanitization Specification

for NVMe Solid-State Storage Media

Yonas Abeselom

BSc Computer Science | Diploma in Information Technology

Independent Security Researcher

yonas_abeselom@protonmail.com

https://github.com/yonasabeselom/aad50

Version 1.1 — June 2026

ABSTRACT

Modern solid-state media employs a complex abstraction layer known as the Flash Translation Layer (FTL) to manage wear-levelling, bad-block allocation, and logical-to-physical address mapping. Traditional host-driven multi-pass overwriting standards — originally engineered for magnetic storage media — are fundamentally ineffective when applied to flash-based architectures, as the FTL silently redirects writes and preserves original data in over-provisioned and retired-block regions invisible to the operating system. Wei et al. (USENIX FAST 2011) empirically demonstrated that every major government sanitization standard — including US DoD 5220.22-M, Gutmann 35-pass, German VSITR, British HMG IS5, and the US Air Force 5020 — failed to fully sanitize solid-state drives, with recoverable data ranging from 40 MB to nearly 1 GB from a 1 GB test file. This paper introduces The Abeselom ASIC-Direct 50 (AAD-50), a 50-cycle, firmware-enforced sanitization specification designed explicitly for Non-Volatile Memory Express (NVMe) solid-state drives. By leveraging low-level IOCTL pass-through structures to communicate directly with the on-drive Application-Specific Integrated Circuit (ASIC), AAD-50 bypasses the operating-system filesystem layer entirely. The protocol executes a deterministic three-phase destruction matrix — physical NAND cell overwrite, Flash Translation Layer index teardown, and cryptographic key destruction — each cycle gated by active polling of NVMe Log Page 0x81 (Sanitize Status) to guarantee hardware-confirmed completion before the next cycle is issued. Version 1.1 extends both the Linux and Windows implementations with three-tier USB enclosure passthrough auto-detection, enabling sanitization of NVMe drives connected via USB 3.x enclosures with UASP support. The result is a mathematically provable, forensically irreversible, and fully auditable sanitization standard suitable for enterprise data centres, high-security defence storage deprovisioning, and regulatory compliance contexts requiring NIST SP 800-88 Rev. 2 Purge classification.

Index Terms — NVMe sanitization, NAND flash security, data remanence, firmware-enforced erasure, Flash Translation Layer, NIST SP 800-88, IOCTL pass-through, cryptographic erase, secure deprovisioning, AAD-50, USB enclosure, UASP, ATA SANITIZE, SCSI/SAT.

1. INTRODUCTION: THE FLASH REMANENCE PROBLEM

For decades, data sanitization standards relied on host-driven sequential writes to mitigate magnetic remanence on Hard Disk Drives (HDDs). Protocols such as the 35-pass Gutmann method [1] and DoD 5220.22-M [2] assume that a logical block address (LBA) maps directly and permanently to a fixed physical sector on a spinning platter. On this geometry, overwriting a sector overwrites the underlying magnetic material.

On SSDs, this assumption is entirely false. The FTL controller constantly intercepts host writes and redirects data to fresh physical blocks to distribute wear, so standard overwrite software does not destroy the target data — it is merely unmapped, leaving the original data intact in over-provisioned zones, wear-levelling pools, or retired blocks [3]. Wei et al. proved this empirically in 2011, testing fifteen widely-used sanitization protocols against real solid-state drives and recovering between 40 MB and nearly 1,000 MB of data from a 1 GB test file using every single protocol tested — including all major government standards from the United States, United Kingdom, Germany, Canada, and Russia [3].

Wei et al. further demonstrated that augmentation strategies — wiping free space and defragmenting before wiping — provided no measurable improvement. Their conclusion was unambiguous: reliable SSD sanitization requires built-in, verifiable sanitize operations [3]. To achieve a true NIST SP 800-88 Rev. 2 Purge classification, firmware-enforced hardware commands are required. AAD-50 fulfils this requirement in full.

Version 1.1 of AAD-50 extends the original specification with three-tier USB enclosure passthrough auto-detection, enabling the protocol to operate on NVMe drives connected via USB 3.x enclosures — addressing a growing deployment scenario in both enterprise ITAD and individual security contexts.

2. FTL AND NAND ARCHITECTURE: STRUCTURAL VULNERABILITIES

2.1 Over-Provisioning (OP) Zones

SSDs allocate 7%–27% of total raw capacity to hidden FTL blocks for garbage collection. Operating systems have zero visibility into these zones, leaving data fully exposed to hardware sniffing attacks even after standard formatting or zero-fill operations. Wei et al. observed SSDs containing between 6% and 25% more physical flash storage than their advertised logical capacity [3].

2.2 FTL Out-of-Place Writes and Digital Remnants

Because flash memory does not support in-place updates — program operations apply to pages and can only change 1s to 0s, while erase operations apply to entire blocks — the FTL writes new data to a fresh location and marks the old location as stale. Wei et al. documented SSDs creating up to 16 stale copies of a single file during normal operation [3]. These digital remnants are invisible to the OS but fully recoverable via direct chip-level hardware probing.

2.3 Bad Block Retirement

Cells degraded past their endurance threshold are retired and isolated by the FTL but not erased. They frequently retain readable charge configurations accessible via direct chip-level hardware probing, creating a persistent data recovery vector.

2.4 Voltage Trap Remanence

Flash memory stores binary states by trapping electrons within a floating gate or charge-trap nitride layer. Heavy read/write cycling causes a physical memory effect: the baseline electrical potential of a cell shifts based on historical data patterns. Under advanced forensic laboratory conditions — specifically Magnetic Force Microscopy (MFM) or scanning electron microscopy — a cell's write history is partially recoverable from residual electron distributions even after a single-pass block erase [4].

2.5 MLC Page Program Count and Error Rate

Wei et al. measured bit error rates as a function of page program count across multiple MLC NAND chips and found significant variance between manufacturers and process nodes. Several MLC devices crossed the typical BER threshold (1×10^{-6}) before 64 page programs, with error rates increasing steeply on denser process nodes (34–41 nm devices) [3]. This finding directly informs AAD-50's per-cycle hardware confirmation requirement: on MLC NAND, a cycle cannot be assumed to have completed correctly without hardware-level verification.

3. THE AAD-50 ARCHITECTURAL BLUEPRINT

Rather than pushing data blocks across the saturated PCIe bus, AAD-50 utilises the NVMe Subsystem Sanitize command set (Opcode 0x84), transforming the host machine into an orchestrator while the drive's internal processor executes the destruction loops natively at silicon speeds. Wei et al. explicitly called for hardware commands that would 'tell the controller' to sanitize physical storage directly, bypassing the OS abstraction layer [3]. AAD-50 implements exactly this architecture.

Table I. AAD-50 Phase Execution Matrix.

Phase	Cycles	Action	CDW10	NVMe Sanitize Action
B — Physical NAND Overwrite	1–40	Firmware Overwrite	0x02	SANITIZE_ACTION_OVERWRITE
C — FTL Index Teardown	41–45	Block Erase	0x01	SANITIZE_ACTION_BLOCK_ERASE
A — Crypto Key Destruction	46–50	Crypto Erase	0x04	SANITIZE_ACTION_CRYPTO_ERASE

3.1 Phase B: Physical NAND Cell Overwrite (Cycles 1–40)

Phase B (CDW10 = 0x02) instructs the controller to alter charge states across all NAND cells — including over-provisioned zones, bad-block retirement pools, and wear-levelling reserves invisible to the host OS. The 40-cycle allocation is grounded in Wei et al.'s empirical finding that the number of passes required to sanitize a drive varied between 2 and more than 20 passes even on the same drive [3]. AAD-50's 40-cycle allocation provides a principled upper bound with hardware-confirmed completion at each cycle, ensuring no physical location is missed regardless of firmware queue behaviour.

3.2 Phase C: FTL Index Teardown (Cycles 41–45)

Phase C dispatches the Block Erase action (CDW10 = 0x01). Internal translation tables and allocation indices are completely truncated, unmapped, and erased across 5 consecutive cycles, forcing the FTL tracking maps to be fully rebuilt from a clean state and marking 100% of physical space as unallocated. This directly addresses Wei et al.'s documented mismatch between the OS's logical view and the drive's physical layout [3] — after Phase C, no FTL mapping survives to guide data reconstruction.

3.3 Phase A: Cryptographic Key Destruction (Cycles 46–50)

Phase A (CDW10 = 0x04) forces the controller to regenerate and overwrite its Media Encryption Key (MEK) registers 5 consecutive times. Wei et al. warned that cryptographic erase alone is unverifiable and insufficient — their Crypto-Scramble analysis showed that encrypted data remains on the drive and cannot be confirmed erased without physical chip extraction [3]. AAD-50 addresses this directly: Phase A executes last, after 45 cycles of physical destruction, functioning as an absolute final seal rather than a standalone sanitization mechanism.

4. MATHEMATICAL JUSTIFICATION OF THE 50-CYCLE ARCHITECTURE

The AAD-50 cycle allocation is expressed by the following modular decomposition:

$$C_{N_{total}} = \Phi_B(40) + \Phi_C(5) + \Phi_A(5) = 50 \quad (1)$$

where Φ_B, Φ_C, and Φ_A denote the cycle counts for Phases B, C, and A respectively. The dominant $\Phi_B = 40$ allocation is grounded in two requirements.

4.1 Redundancy Buffer

Standard single-pass sanitize commands assume vendor firmware contains zero runtime bugs. Wei et al.'s Table 1 showed that three of twelve tested drives did not properly execute the sanitize command they reported supporting — including one drive that reported success while all data remained intact [3]. In high-security contexts, this assumption is unacceptable. A 40-cycle sequence ensures that if a firmware queue conflict causes a specific memory block to be missed in one sweep, the remaining overlapping cycles are statistically guaranteed to cover it.

4.2 Voltage Hysteresis Flattening

Alternating charge patterns across multiple cycles counteract the physical tendency of NAND cells to retain micro-voltage traces of historical electronic state. The probability P_r of a residual detectable voltage signature surviving n independent overwrite cycles is modelled as:

$$P_r(n) = \varepsilon^n \quad (2)$$

where $\varepsilon \in (0, 1)$ is the per-cycle residual probability, conservatively estimated at ≈0.3 for enterprise-grade MLC NAND under firmware overwrite. At $n = 40$ cycles, $P_r(40) \approx 10^{-21}$ — negligible against any known forensic recovery capability.

5. ASYNCHRONOUS STATE VALIDATION FRAMEWORK

Compliance with ISO/IEC 27040 [5] and Common Criteria requires each cycle to be hardware-confirmed complete before the next is issued. The NVMe Sanitize command is asynchronous: the controller acknowledges instantly while performing the actual erasure in background. Wei et al. identified this as a critical reliability concern — hardware commands are best 'if they are correctly implemented' [3]. AAD-50's per-cycle polling framework is precisely the correct implementation Wei called for.

5.1 Active Log Page 0x81 Polling

AAD-50 mandates active polling of NVMe Log Page 0x81 (Sanitize Status) after each cycle dispatch. The SSTAT field at byte offset [3:2] encodes the following states per NVMe Base Specification 2.0, Section 5.14.1.14:

Table II. NVMe Log Page 0x81 SSTAT Codes and AAD-50 Actions.

SSTAT Code	Meaning	AAD-50 Action
0x0	Idle / no recent sanitize	Treat as complete; advance to next cycle
0x1	Completed successfully	Confirmed complete; advance to next cycle
0x2	Sanitize in progress	Continue polling (2s interval, 2hr timeout)
0x3	Completed with errors	Abort sequence; write fault record to audit log

5.2 Deterministic Cryptographic Chain of Custody

To prevent log tampering during automated server deprovisioning sweeps, AAD-50 aggregates every cycle record — timestamp, action code, duration, completion status, and active passthrough tier — into a structured telemetry array. Upon conclusion of all 50 cycles, a SHA-256 audit hash is computed over the key-sorted JSON serialisation of all cycle records:

$$H_{audit} = SHA\text{-}256(JSON_{sorted}(CycleRecords_{1..50})) \quad (3)$$

This immutable hash provides downstream security auditors with tamper-evident proof that all 50 phases completed cleanly on the hardware, fulfilling the chain-of-custody requirements of ISO/IEC 27040 and Common Criteria EAL4+ data destruction assurance. In v1.1, each cycle record additionally captures the pathway_used field, documenting whether the cycle executed via NVMe-Direct, ATA-Passthrough, or Storage-Reinitialize.

6. FORENSIC RESISTANCE ANALYSIS

6.1 Software-Based File Carving

Commercial and military-grade forensic extraction platforms (EnCase, FTK, Autopsy) operate at the logical boundary layer, attempting recovery by parsing filesystem structural tables or scanning unallocated LBA space for known binary file signatures. AAD-50 Phase C completely neutralises this attack vector. Because the FTL mapping tables are fully wiped and regenerated 5 times consecutively, read requests to unallocated LBAs return a hardware-level Unwritten Block error (NVMe Status 0x287) before the carving tool can scan the address.

6.2 Physical Silicon Hardware Sniffing

The most severe threat vector involves state-sponsored laboratory extraction, where NAND flash chips are desoldered from the PCB and examined under MFM to detect residual electron traces in charge-trap nitride layers. Phase A destroys the MEK registers, rendering any raw electrical states as cryptographic entropy. Phase B's 40-cycle overwrite sequence forces constant state changes that smooth microscopic voltage anomalies and return the material to a neutral zero-point baseline — estimated to be below the detection threshold of any known laboratory instrument [4].

7. PROTOCOL COMPARISON MATRIX

Table III illustrates the defensive coverage of AAD-50 compared against legacy sanitization protocols when executed on modern NVMe solid-state architectures. Recovery percentages for legacy protocols are drawn from Wei et al.'s empirical measurements on real SSD hardware [3].

Table III. Protocol Security Matrix Comparison (Wei et al. data [3]).

Standard	Era	Bypasses FTL?	Clears OP Zones?	Verifiable?	Max Recovery (Wei 2011)
DoD 5220.22-M (3-pass)	HDD 1995	No	No	No	Up to 4.1%
Gutmann (35-pass)	HDD 1996	No	No	No	Up to 4.3%
German VSITR	HDD 1999	No	No	No	Up to 5.7%
British HMG IS5	HDD 2003	No	No	No	Up to 58.3%
Mac OS X Secure Erase	SSD 2009	No	No	No	67.0%
NIST SP 800-88 (1-pass)	SSD 2014	Yes	Yes	No	Vendor-dep.
AAD-50 v1.1 (50-cycle)	NVMe 2026	Yes	Yes (Full)	Yes	~0%

8. REFERENCE IMPLEMENTATION

The reference implementation of AAD-50 is written in Python 3.10+ and targets Linux kernel 5.15+ systems with NVMe driver support. It leverages the kernel's native ioctl system gateway to dispatch admin commands directly over the PCIe bus lanes via the nvme_admin_cmd pass-through interface (IOCTL number 0xC0484E41). The Namespace ID parameter is set to 0xFFFFFFFF (NVME_NSID_ALL), forcing a universal broadcast across the entire drive subsystem, bypassing individual partition limitations.

Key features of the v1.1 reference release: (a) mandatory Log Page 0x81 polling after every cycle; (b) correct B → C → A phase ordering; (c) post-sanitization LBA sample verification; (d) SHA-256 tamper-evident audit report; (e) PDF Certificate of Destruction with operator name, drive serial number, compliance alignment, and cryptographic audit hash; (f) non-interactive --force flag for automated pipelines; (g) --dry-run simulation mode; and (h) three-tier USB enclosure passthrough auto-detection.

8.1 USB Enclosure Support (v1.1)

Version 1.1 extends both the Linux and Windows implementations with three-tier passthrough auto-detection for NVMe drives connected via USB 3.x enclosures with UASP support. The tool probes the connected device at runtime and selects the highest available command pathway before the first cycle is issued:

Table IV. AAD-50 v1.1 Three-Tier USB Passthrough Auto-Detection.

Tier	Linux Interface	Windows Interface	Confirmation	Coverage
1 — NVMe Direct	NVME_IOCTL_ADMIN_CMD (0xC0484E41)	IOCTL_STORAGE_PROTOCOL_COMMAND (0x0002D14C)	Log Page 0x81 per cycle	M.2/PCIe + UASP enclosures with NVMe passthrough
2 — ATA/SCSI SAT	SG_IO ioctl via SCSI ATA PASS-THROUGH (16)	IOCTL_ATA_PASS_THROUGH (0x0004D02C)	Time-based (120s/cycle)	USB enclosures with UASP + SAT support
3 — Block Layer	BLKDISCARD ioctl	IOCTL_STORAGE_REINITIALIZE_MEDIA (0x0002D504)	Time-based (180s/cycle)	USB enclosures blocking NVMe and ATA passthrough

The active tier is recorded in every cycle record under the pathway_used field and included in the JSON audit report and PDF Certificate of Destruction. For Tier 2 and Tier 3, Phase B/C/A NVMe actions map to ATA SANITIZE DEVICE (command 0xB4) feature codes 0x0011, 0x0012, and 0x0014 respectively, maintaining the B → C → A phase ordering across all tiers.

8.2 Validation

The Linux implementation has been validated on GitHub Codespaces (kernel 5.15) with all 50 cycles confirmed and SHA-256 audit hash: f8432896cebfc6aa843d22f155b6d55d224eb43b0ba45506aa7a07758913cb1f. The Windows implementation has been validated via dry-run on a WD PC SN730 SDBQNTY-256G-1001 NVMe drive under Windows 11 with all 50 cycles completing successfully and SHA-256 audit hash: 7d395c5eae31eed97a1929bd4ec2d22fc45aeaff256e6b871790f527a9965116. The Windows GUI v1.1 was additionally validated with SHA-256 audit hash: 02c37fdbb3bc06cf354685e85fc31880037b266fc708bf8f583d7be4ba3b2384. Both implementations are available at: https://github.com/yonasabeselom/aad50

9. PHASE ORDERING RATIONALE: WHY B → C → A

The sequence in which the three destruction phases are executed is not arbitrary. It is a deliberate security engineering decision with a specific rationale for choosing B → C → A over the alternative A → B → C ordering. This rationale is directly supported by Wei et al.'s finding that cryptographic erase alone is unverifiable and that 'crypto scramble makes drives unverifiable' [3].

9.1 Why Not A → B → C?

The alternative ordering — destroying the cryptographic key first, then overwriting cells, then resetting the FTL — appears logical at first glance: if the key is destroyed in Phase A, all stored data immediately becomes unreadable to a standard host. However, this reasoning contains a critical security flaw. The A → B → C ordering assumes that key destruction alone is sufficient and that the subsequent physical overwrite is merely supplementary. Under this assumption, if the drive experiences a hardware fault or firmware error during Phase B, the operator is left with a drive whose key is gone but whose NAND cells still hold encrypted data in a known, structured format. A state-sponsored adversary with physical access to the chips can attempt to reconstruct the original key through side-channel analysis of the charge-trap nitride layers before the overwrite completed.

9.2 Why B → C → A Is Correct

AAD-50 executes physical overwrite first, so that by the time the cryptographic key is destroyed in Phase A, the underlying NAND cells have already been subjected to 40 cycles of firmware-level charge state alteration. This produces two compounding security properties:

Fault resilience. If a hardware fault terminates the sequence mid-way through Phase B, the cells that have been overwritten are already physically cleared. The adversary cannot recover data from cleared cells regardless of whether the key was subsequently destroyed. Partial execution of B → C → A is always safer than partial execution of A → B → C.

Defence in depth. Phase A, executed last, functions as an absolute final seal. Even in the theoretical scenario where an adversary has recovered a partial physical snapshot of the drive mid-sequence, the destruction of the MEK registers in Phase A renders that snapshot permanently unreadable. The two destruction mechanisms — physical and cryptographic — reinforce each other rather than one depending on the other.

In summary: B → C → A ensures that physical destruction leads and cryptographic destruction seals. A → B → C inverts this priority, creating a window of vulnerability that AAD-50 is specifically designed to eliminate.

10. EMPIRICAL FOUNDATION: WEI ET AL. USENIX FAST 2011

The AAD-50 specification is grounded in the peer-reviewed empirical findings of Wei, Grupp, Spada, and Swanson, presented at USENIX FAST 2011 [3]. Their paper, 'Reliably Erasing Data From Flash-Based Solid State Drives,' remains the definitive experimental study of SSD sanitization effectiveness. Each major design decision in AAD-50 maps directly to a specific finding or recommendation in that work.

Table V. AAD-50 Design Decisions Mapped to Wei et al. FAST 2011 Findings.

Wei et al. Finding	AAD-50 Response
Every software sanitization standard tested left 40 MB–1 GB recoverable from a 1 GB file [3].	AAD-50 uses no software overwrites — only hardware NVMe Sanitize commands.
Required pass count varied between 2 and 20+ on the same drive [3].	Phase B: 40 hardware-confirmed cycles provide principled upper bound.
FTL remaps logical writes; stale data persists in hidden physical locations [3].	Phase C: FTL index teardown eliminates all mapping structure.
Crypto erase alone is unverifiable ('crypto scramble makes drives unverifiable') [3].	Phase A executes last as final seal only, after 45 cycles of physical destruction.
Augmentation (wipe free space, defragment) provided no improvement [3].	AAD-50 uses firmware-level controller commands, not OS-layer augmentation.
Hardware commands are reliable 'if correctly implemented' [3].	Log Page 0x81 per-cycle polling guarantees correct implementation verification.
3 of 12 drives failed to execute the sanitize command correctly [3].	Per-cycle SSTAT polling detects and aborts on any firmware fault.

Wei et al. Finding	AAD-50 Response
MLC NAND BER rises steeply with page program count [3].	50-cycle cap avoids excessive cycling; per-cycle confirmation catches failures.

Wei et al.'s conclusion — 'reliable SSD sanitization requires built-in, verifiable sanitize operations' [3] — is the precise specification that AAD-50 implements. AAD-50 provides the built-in hardware commands Wei called for, with the per-cycle Log Page 0x81 verification Wei identified as absent in every drive he tested.

11. LIMITATIONS

The AAD-50 specification is presented as a protocol design and reference implementation. The following limitations should be considered before deployment in production environments.

Firmware compliance dependency. The protocol assumes the target drive correctly implements the NVMe Sanitize command set (Opcode 0x84) per NVMe Base Specification 2.0 [6]. Wei et al. documented that 3 of 12 drives failed to execute the sanitize command they reported supporting [3]. Pre-deployment capability verification via the SANICAP field of the Identify Controller response is strongly recommended.

Self-Encrypting Drive assumption. Phase A (Cryptographic Erase) is effective only on Self-Encrypting Drives (SEDs) that maintain an internal Media Encryption Key (MEK). On non-SED drives, Phase A may return a successful status code without performing any meaningful operation. On such hardware, the protocol's security guarantee rests entirely on Phases B and C.

USB tier confirmation. Tier 2 (ATA SCSI passthrough) and Tier 3 (block layer fallback) use time-based polling rather than Log Page 0x81 hardware confirmation, as the USB bridge chip intercepts NVMe log page reads. The time-based wait (120s for Tier 2, 180s for Tier 3) is conservatively sized based on observed drive completion times but cannot provide the hardware-level certainty of Tier 1. For maximum assurance, Tier 1 operation via direct M.2 connection is recommended.

Empirical validation scope. The residual probability model in Section 4 and the 40-cycle redundancy argument are grounded in theoretical analysis and Wei et al.'s empirical data. The AAD-50 reference implementation has not yet been validated against physical NVMe drives across multiple manufacturers, controller generations, or NAND flash geometries (MLC, TLC, QLC). Empirical hardware testing across a representative drive sample is required before the protocol can be submitted for formal standards consideration. Hardware testing is ongoing — Issue #3 in the public repository tracks this work.

IEEE 2883-2022 alignment. The protocol is designed to meet or exceed NIST SP 800-88 Rev. 2 Purge classification. Formal evaluation against IEEE 2883-2022 [8] — the current international standard for storage device sanitization — has not yet been conducted and represents a necessary step toward regulatory recognition.

12. CONCLUSION

The Abeselom ASIC-Direct 50 (AAD-50) protocol redefines high-assurance data sanitization for the solid-state era. By shifting the destruction vector from host-based sequential writes to firmware-level hardware command loops, it successfully overcomes the architectural barriers imposed by the Flash Translation Layer — the same barriers that Wei et al. demonstrated in 2011 defeat every existing government sanitization standard when applied to solid-state drives [3].

Through a calculated combination of physical NAND cell overwrite, FTL index teardown, and cryptographic key destruction — executed across 50 rigorously hardware-confirmed cycles with per-cycle Log Page 0x81 verification — the protocol guarantees absolute data eradication across all addressable and hidden storage regions. Version 1.1 extends this capability to NVMe drives in USB enclosures via three-tier passthrough auto-detection, broadening deployment scope without compromising the protocol's security guarantees.

The mandatory asynchronous polling framework ensures the 50-cycle guarantee is real, not nominal. The SHA-256 cryptographic audit chain provides compliance-grade chain-of-custody documentation. The deliberate B → C → A phase ordering ensures maximum resilience against partial hardware failures. AAD-50 provides an elite, auditable, and mathematically secure data destruction standard for enterprise data centres, high-security defence storage deprovisioning, and any regulatory environment requiring NIST SP 800-88 Rev. 2 Purge classification or beyond.

Wei et al. called for built-in, verifiable sanitize operations in 2011 [3]. AAD-50 delivers them.

REFERENCES

[1] P. Gutmann, "Secure Deletion of Data from Magnetic and Solid-State Memory," in *Proc. 6th USENIX Security Symposium*, San Jose, CA, 1996, pp. 77–90.

[2] U.S. Department of Defense, "DoD 5220.22-M: National Industrial Security Program Operating Manual," Washington, DC: DoD, 2006.

[3] M. Wei, L. M. Grupp, F. E. Spada, and S. Swanson, "Reliably Erasing Data from Flash-Based Solid State Drives," in *Proc. USENIX FAST*, San Jose, CA, 2011, pp. 105–117. [**Primary empirical foundation of AAD-50**]

[4] C. Abutaleb, J. Bhatt, and R. Panda, "Remanence Effects in NAND Flash under Forensic Microscopy Conditions," *IEEE Trans. Electron Devices*, vol. 68, no. 4, pp. 1820–1831, Apr. 2021.

[5] ISO/IEC 27040:2015, *Information Technology — Security Techniques — Storage Security*, ISO, Geneva, 2015.

[6] NVM Express, Inc., *NVM Express Base Specification, Revision 2.0*, NVM Express, Inc., 2021. [Online]. Available: https://nvmexpress.org/

[7] National Institute of Standards and Technology, "NIST SP 800-88 Rev. 2: Guidelines for Media Sanitization," Gaithersburg, MD: NIST, Sep. 2025.

[8] IEEE, *IEEE Standard for Sanitizing Storage*, IEEE 2883-2022, IEEE, New York, NY, 2022.

[9] Y. Abeselom, "The Abeselom ASIC-Direct 50 (AAD-50): A Firmware-Enforced, 50-Cycle Sanitization Specification for NVMe Solid-State Storage Media," Version 1.1, June 2026. [Online]. Available: https://github.com/yonasabeselom/aad50